AI-Powered Resume Ranker – Project Report

1. Introduction

In the modern recruitment process, organizations often receive hundreds or even thousands of resumes for a single job position. Manually screening these resumes is time-consuming, inefficient, and prone to human bias. To address these challenges, automated systems are required to streamline the hiring process and ensure fair evaluation.

The AI-Powered Resume Ranker is developed as a solution to automate resume screening using Natural Language Processing (NLP) and Machine Learning techniques. The system evaluates resumes based on their relevance to a given job description and ranks candidates accordingly.

By leveraging text processing, similarity analysis, and skill-based scoring, the system helps recruiters quickly identify the most suitable candidates. This project demonstrates the practical application of AI in solving real-world problems in recruitment and human resource management.

2. Abstract

The AI-Powered Resume Ranker is a web-based application that automates the process of evaluating and ranking resumes based on a job description. The system extracts text from PDF resumes, preprocesses the content using NLP techniques, and converts the text into numerical vectors using TF-IDF (Term Frequency–Inverse Document Frequency).

Cosine similarity is used to measure the relevance between each resume and the job description. In addition to similarity scoring, the system incorporates a skill-based scoring mechanism to enhance ranking accuracy by identifying key technical skills within resumes.

A weighted scoring method is applied to combine similarity and skill scores, resulting in a final score for each candidate. The resumes are then ranked in descending order based on this score. The application provides a user-friendly interface for uploading resumes, entering job descriptions, viewing ranked results, and downloading reports.

This project highlights the effectiveness of NLP and machine learning techniques in automating recruitment processes and improving efficiency.

3. Tools Used

The following tools and technologies were used to develop the project:

Programming Language

- Python 3.10

Libraries and Frameworks

- Flask – Used to develop the backend web application and manage routing and file handling
- SpaCy – Used for Natural Language Processing tasks such as tokenization, lemmatization, and stop-word removal
- Scikit-learn – Used for TF-IDF vectorization and cosine similarity computation
- PyMuPDF (fitz) – Used to extract text from PDF resumes
- Pandas – Used for data manipulation and generating CSV reports
- NumPy – Used for numerical computations

Frontend Technologies

- HTML – Structure of web pages
- CSS – Styling and layout of the user interface

Development Tools

- Visual Studio Code – Code editor
- Git – Version control (optional)

4. Steps Involved in Building the Project

The project was developed through the following steps:

Step 1: Data Input
Users upload resumes in PDF format and provide a job description through the web interface.
Step 2: Text Extraction
Text is extracted from PDF resumes using the PyMuPDF library. This converts unstructured data into readable text format.
Step 3: Text Preprocessing
The extracted text is cleaned and processed using SpaCy:

- Conversion to lowercase
- Removal of stop words
- Lemmatization of words
- Removal of non-alphabetic tokens

This ensures meaningful data representation.
Step 4: Feature Extraction (TF-IDF)
The processed text is converted into numerical vectors using TF-IDF. This helps in identifying important terms in the documents relative to others.
Step 5: Similarity Calculation
Cosine similarity is used to calculate how closely each resume matches the job description. This produces a similarity score for each resume.
Step 6: Skill-Based Scoring
A predefined list of relevant skills is used to identify key skills present in each resume. A skill score is calculated based on the number of matching skills.
Step 7: Final Scoring and Ranking
A weighted scoring formula is applied:

- 70% weight to similarity score
- 30% weight to skill score

Resumes are then ranked based on the final score.
Step 8: Web Application Development
A Flask-based web application is developed to:

- Upload resumes
- Input job descriptions
- Display ranked results
- Highlight top candidates

Step 9: Report Generation
The final ranked results are stored in a CSV file using Pandas, allowing users to download and analyze the results.

5. Conclusion

The AI-Powered Resume Ranker successfully demonstrates how Natural Language Processing and Machine Learning techniques can be applied to automate the resume screening process. The system improves efficiency by reducing manual effort and provides a fair and consistent method for evaluating candidates.
The use of TF-IDF and cosine similarity ensures effective text comparison, while the inclusion of skill-based scoring enhances the accuracy of rankings. The user-friendly web interface makes the system practical and easy to use.
In the future, the system can be enhanced by integrating advanced NLP models such as BERT for better semantic understanding, implementing Named Entity Recognition (NER) for detailed resume parsing, and deploying the application on cloud platforms for scalability.